Linux First Class Notes

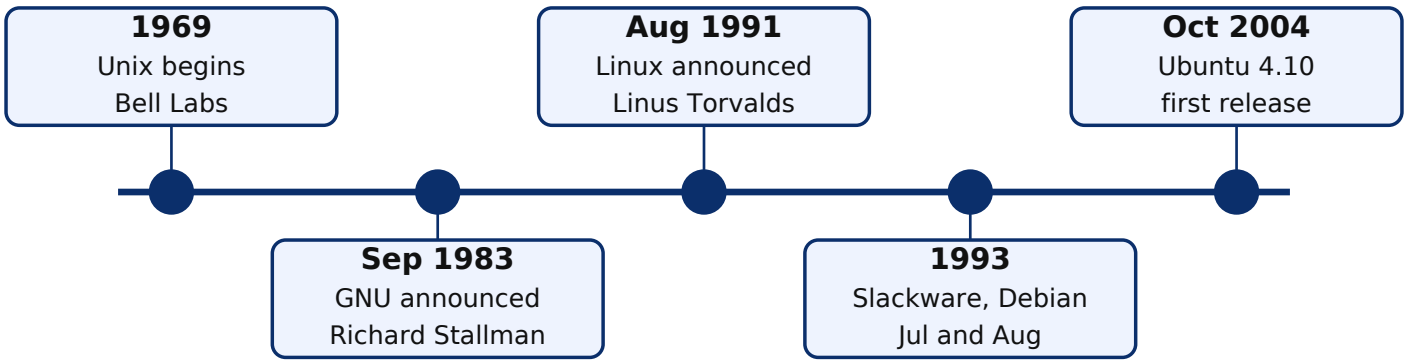
History, distributions and the basic file commands - with examples, command chart and interview Q&A

How to read this: examples need no sudo. Everything here works on files in your own home folder as a normal user. `OWN FILES` = safe in your own folder. `DELETES` = removes things with no undo. Replace words in <angle brackets> with real names.

1. What is Linux? A short history

Linux is a free, open-source operating system family. Strictly, "Linux" is the **kernel** - the core that talks to hardware. A **distribution** (distro) packages that kernel with a shell, commands, an installer and a package manager so you can actually use it.

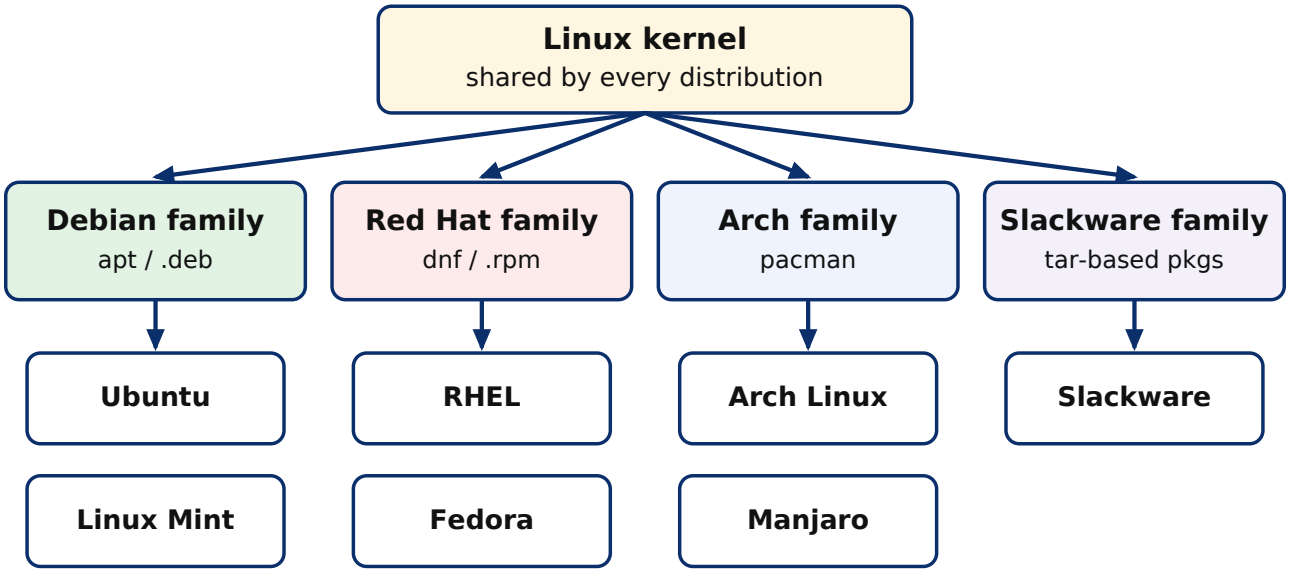
Linux grew out of **Unix**, which began at Bell Labs in 1969 with Ken Thompson and Dennis Ritchie. On 27 September 1983 Richard Stallman announced the **GNU** project, a plan for a free Unix-like system (work officially began in January 1984). On 25 August 1991 a student, **Linus Torvalds**, announced on Usenet that he was making a free operating system; version 0.01 of his kernel was released on 17 September 1991. GNU tools plus the Linux kernel together form the complete free system.



Correction to remember: Linus Torvalds started the Linux **kernel**, not all of Linux-the-system and not Unix. Dennis Ritchie made C and co-created Unix. Dates in the timeline are from the sources listed on the last page.

2. Linux distributions

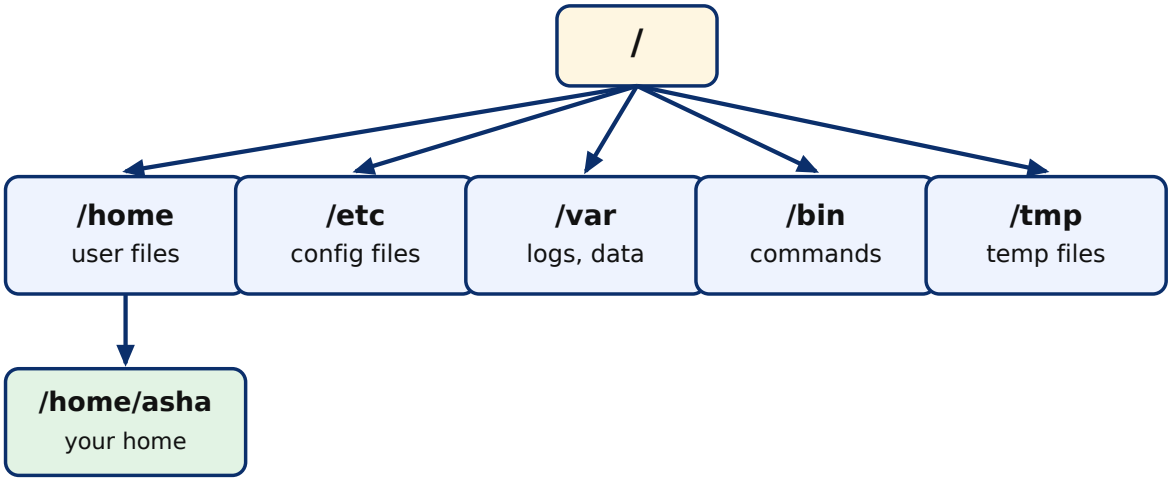
A distribution is a ready-made Linux. They differ in who maintains them, how often they update, and which **package manager** installs software. The file commands in these notes are the same on all of them; only install commands differ (apt vs dnf). Ubuntu or Debian is the usual start.



Family	Known for	Package tool
Debian	Community project begun in August 1993 by Ian Murdock. Ubuntu is built from Debian.	`apt`, `.deb` files
Ubuntu	First release (4.10, "Warty Warthog") on 20 October 2004, announced by Mark Shuttleworth. Made by Canonical.	`apt`
Red Hat family	RHEL is Red Hat's enterprise distribution with subscription support. Fedora is the community distribution upstream of RHEL.	`dnf`, `.rpm` files
Arch	Rolling release with latest stable software. Install gives only a command-line base; you choose the packages. Aimed at users willing to read the documentation, so the learning curve is steeper.	`pacman`
Slackware	Slackware 1.00 was announced on 16 July 1993 and the distribution is still maintained.	its own tools

3. Files, directories and paths

Linux has one tree that starts at **/** (called root). Everything - files, folders, even devices - hangs from it. Your own space is your **home directory**, usually `/home/<you>`.



Path shortcuts

Symbol	Meaning	Example
`/`	The top of the tree; also the separator between folders	`/home/asha/notes.txt`
`~`	Your home directory	`~/notes.txt`
`.`	The current directory	`cp notes.txt .`
`..`	The parent (one level up)	`cd ..`
`.name`	A name starting with a dot is a **hidden** file	`.bashrc`

Absolute path starts with `/` and works from anywhere. **Relative path** starts from where you are now. Names are **case-sensitive**: `Notes.txt` and `notes.txt` are different files.

4. Basic commands with examples

Each card shows what the command does, an example, and what to watch for. Start the practice from your home folder.

pwd - where am I? ANY USER

Prints the full path of the current directory.

```
pwd
# /home/asha
```

cd - change directory ANY USER

```
cd Documents      # go into Documents (relative)
cd /etc           # absolute path
cd ..             # one level up
cd ~              # home (same as plain: cd)
cd -              # back to the previous directory
```

cd only moves you; it creates nothing. If the folder does not exist you get "No such file or directory".

ls - list what is in a directory ANY USER

```
ls                # names only (hidden files not shown)
ls -a             # all: includes hidden files, plus . and ..
ls -l             # long format: details for each item
ls -al            # long format AND hidden files (same as -la)
ls -lh            # sizes like 4.0K, 2.1M (human readable)
ls Documents      # list another directory without cd
```

Options can be combined in one word: `-al` = `-a` + `-l`. The order does not matter.

Reading ls -l output

-	rw-r--r--	1	asha	asha	2048	Oct 26 14:20	notes.txt
type	permissions	links	owner	group	size (bytes)	modified	name

First character	Meaning
-	regular file
d	directory
l	symbolic link (shortcut)

The next nine characters are three triplets for owner, group and others: r read, w write, x execute. A dash means "not allowed".

Command	Shows
ls	Names only, hidden files skipped
ls -a	Names including hidden (. and .. too)
ls -A	Hidden files but not . and ..
ls -l	Permissions, owner, size, date - hidden skipped
ls -al	Everything: details and hidden files
ls -R	Lists sub-directories recursively
ls -t	Newest first (add -r to reverse)

mkdir - make a directory OWN FILES

```
mkdir project              # one folder
mkdir a b c                # several at once
mkdir -p work/2026/oct     # create parent folders too
```

Without -p, creating work/2026/oct fails if work does not exist.

touch - create an empty file OWN FILES

```
touch notes.txt            # create empty file
touch a.txt b.txt c.txt    # create several files
touch notes.txt            # file exists: only updates its time
```

If the file already exists, **touch** does not erase it - it just updates the modified time. Brace shortcut: `touch file{1..3}.txt` makes three files.

cp - copy files and directories OWN FILES

```
cp notes.txt backup.txt        # copy to a new name
cp notes.txt Documents/        # copy into a folder
cp -r project project_copy     # copy a directory and contents
cp -i notes.txt backup.txt     # ask before overwriting
cp -v a.txt b.txt              # say what was copied
```

Warning: if the target name already exists, cp overwrites it silently unless you use `-i` (ask) or `-n` (never overwrite). Copying a directory needs `-r`. The original stays.

mv - move or rename OWN FILES

```
mv old.txt new.txt          # rename
mv notes.txt Documents/      # move into a folder
mv a.txt b.txt Documents/    # move several files
mv -i a.txt b.txt            # ask before overwriting
```

There is no separate rename command: **renaming is moving to a new name**. Unlike cp, the original is gone from the old place. `mv` also overwrites silently without `-i`.

Copy and move, step by step (class demonstration)

Root demonstration - watch and understand, do not run it. These steps were run as the administrator (`root`) on the class machine, using `/root` (the private home of the root user) and `/opt`. A normal user cannot enter `/root`. Practise the same ideas on your own files, or in the safe lab in section 5.

```
[root@localhost opt]# ls -l /opt    # 1. check what is in /opt
total 4
-rw-r--r--. 1 root root 29 Oct 13 07:35 3.txt   # already here
[root@localhost opt]# cp /root/3.txt /opt/test.txt   # 2. copy
[root@localhost opt]# ls -l /opt    # 3. verify: both listed
total 8
-rw-r--r--. 1 root root 29 Oct 13 07:35 3.txt
-rw-r--r--. 1 root root 29 Oct 13 07:38 test.txt  # new copy
[root@localhost opt]# cat /opt/test.txt   # 4. check contents
[root@localhost opt]# mv /root/3.txt /opt/   # 5. move into /opt
mv: overwrite '/opt/3.txt'? y   # it existed: asked first
```

Reading the steps: the teacher checked `/opt` first (a `3.txt` was already there). Step 2 copies the file to the **new name** `test.txt` - the original stays in `/root`. Steps 3 and 4 verify the copy: `ls -l` shows it with the same size, `cat` shows the same contents. Step 5 **moves** the file into the existing folder `/opt/`: it keeps its name and disappears from `/root`. Because `/opt/3.txt` already existed, the machine asked before replacing it.

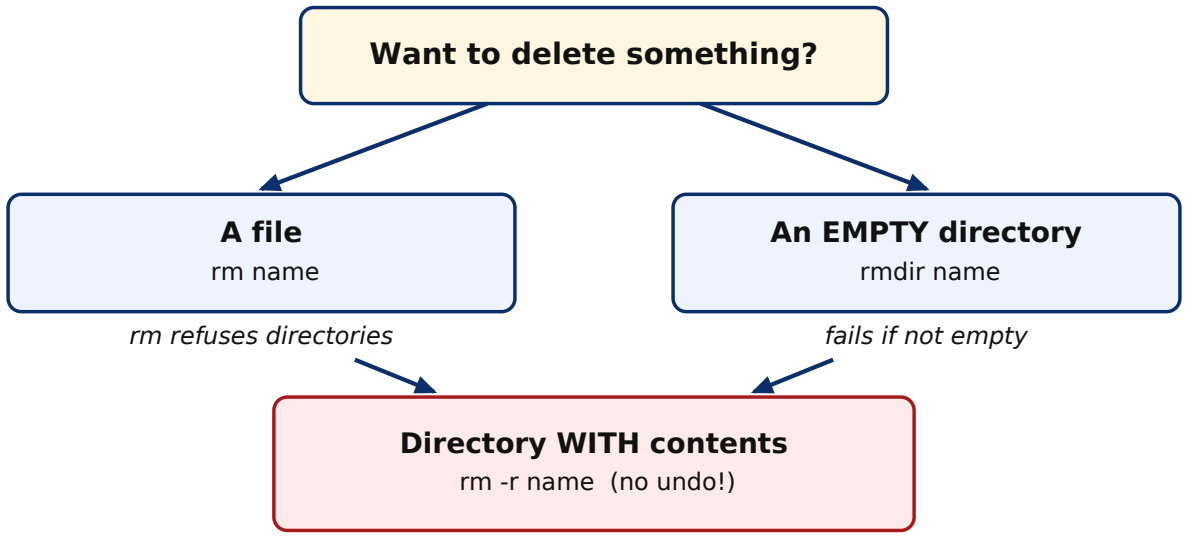
Destination you type	What happens
An existing folder, e.g. `/opt/`	The file keeps its name and lands inside: `/opt/3.txt`
A new name, e.g. `/opt/test.txt`	The copy (or the moved file) gets that new name
A name that already exists	It is replaced - see the overwrite rules below
The SOURCE is a folder	cp refuses without `-r` ("omitting directory" error); `mv` moves the whole folder with no extra option

```
cp -r /root/myfolder /opt/   # copy a WHOLE folder (needs -r)
mv /root/myfolder /opt/      # move a whole folder (no -r)
```

When does it ask "overwrite ...?": By default `cp` and `mv` replace an existing file **silently**. `-i` asks before each replacement; `-n` never overwrites. The prompt can also come from a shell alias that adds `-i` automatically (common for the root user on Red Hat family systems) - the photo does not show which one the class machine used. Check your own shell with `alias cp` and `alias mv`, and never rely on the prompt.

5. Remove and delete

There is no recycle bin on the command line. Deleted files do not go to Trash; assume they are gone for good. Check with `pwd` and `ls` before you delete.



rm - remove files DELETES

```
rm old.txt              # delete one file
rm a.txt b.txt          # delete several
rm -i old.txt           # ask "remove?" first (safest habit)
rm -v old.txt           # print what was removed
rm -r project           # delete a directory and EVERYTHING in it
rm -f missing.txt       # force: no prompts, ignore missing files
```

By default `rm` does not remove directories. `-r` (recursive) deletes a folder and all its contents. `-f` removes the safety prompts. **Never** combine them with a path you have not checked, for example `rm -rf` with a typo, a variable that is empty, or `/`.

rmdir - remove an empty directory DELETES

```
rmdir emptyfolder
rmdir -p a/b/c          # removes c, b, a if all empty
```

`rmdir` is the safe way to delete a directory: it refuses if the directory still has files. If it complains "Directory not empty", look with `ls -a` (hidden files count).

Practice lab (safe: a brand-new temporary folder)

Type these **one at a time, not as one pasted block**. `mktemp -d` makes a new, empty folder with a unique name, so it cannot clash with your existing files. Before any delete step, run pwd and check it shows that temporary folder.

```
d=$(mktemp -d)                # new empty folder, unique name
cd "$d"                       # enter it
pwd                           # CHECK: shows the new folder
touch a.txt b.txt c.txt       # create three files
ls -l                         # see them with details
cp a.txt copy_of_a.txt        # copy
mv b.txt renamed_b.txt        # rename
mkdir archive
mv c.txt archive/             # move into archive
cp -r archive archive_copy    # copy the whole folder
ls -R                         # see archive and its copy
```

Delete steps - run pwd first and only continue if you are still in the temporary folder:

```
rm -i copy_of_a.txt           # asks first (answer y)
rmdir archive                 # fails: not empty
rm -ri archive archive_copy   # asks about each item, removes both
cd .. && rmdir "$d"           # remove the empty temporary folder
```

If any step shows an error, stop and check pwd before typing the next one.

6. Command chart

Command	What it does	Key options	Safe?
`pwd`	Show current directory	-	safe
`cd <dir>`	Change directory	`..` up, `~` home, `-` previous	safe
`ls`	List contents	`-a` hidden, `-l` details, `-al` both, `-h` sizes, `-R` recursive, `-t` by time	safe
`mkdir <dir>`	Create directory	`-p` create parents	safe
`touch <file>`	Create empty file / update time	several names at once	safe
`cp <src> <dst>`	Copy (original stays)	`-r` needed for directories, `-i` ask, `-n` no overwrite, `-v` verbose	overwrites silently
`mv <src> <dst>`	Move or rename (folders too, no -r)	`-i` ask, `-n` no overwrite, `-v` verbose	overwrites silently
`rm <file>`	Delete files	`-i` ask, `-r` directories, `-f` force, `-v` verbose	DELETES
`rmdir <dir>`	Delete empty directory	`-p` parents too	DELETES (empty only)
`cat <file>`	Print a file	`-n` number lines	safe
`echo "text"`	Print text; with `>` writes to a file	`>` overwrite, `>>` append	`>` overwrites
`man <cmd>`	Open the manual page	`q` to quit	safe
`clear`	Clear the screen	-	safe

Safety habit: when unsure, run `ls` first and add `-i` to `cp`, `mv` and `rm`.

7. Interview questions and answers

Q1. What is Linux, and who created it?

Linux is a free, open-source Unix-like operating system. Linus Torvalds started the Linux kernel in 1991. Combined with GNU tools it became a complete system. Strictly, "Linux" is the kernel; distributions package it into a usable OS.

Q2. What is a distribution? Name a few.

A distribution is the kernel plus tools, an installer and a package manager. Examples: Debian, Ubuntu (Debian family), RHEL and Fedora (Red Hat family), Arch, Slackware.

Q3. How do Debian-based and Red Hat-based distros differ in practice?

Mainly in packaging: Debian family uses .deb files and apt; Red Hat family uses .rpm files and dnf. The core file commands are the same.

Q4. What is the difference between Unix and Linux?

Unix came first, built at Bell Labs in 1969. Linux is a separate, free kernel that follows Unix ideas; it is Unix-like, not Unix itself.

Q5. What does pwd do?

It prints the full path of the directory you are currently in.

Q6. What is the difference between absolute and relative paths?

An absolute path starts at / and works from anywhere (/home/asha/notes.txt). A relative path starts from the current directory (Documents/notes.txt).

Q7. What do ls, ls -a, ls -l and ls -al do?

ls lists names. -a includes hidden files (starting with a dot). -l shows details: permissions, owner, group, size, date. -al does both together; -la is the same.

Q8. How can you tell a file from a directory in ls -l?

By the first character of the line: - for a regular file, d for a directory, l for a symbolic link.

Q9. What are . and ..?

. is the current directory and .. is its parent. cd .. moves up one level.

Q10. How do you create files and directories?

touch creates empty files (and updates the time of existing ones). mkdir creates directories; mkdir -p creates parent folders too.

Q11. How do you copy a directory?

cp -r source destination. Without -r, cp refuses a directory. cp overwrites an existing target silently, so use -i to be asked.

Q12. How do you copy a file into another directory? With a new name?

cp file /dir/ keeps the same name inside /dir. cp file /dir/newname.txt saves the copy under a new name. Either way the original stays in place.

Q13. Does mv need -r for directories?

No. mv moves or renames a whole directory as it is. Only cp needs -r, because copying a directory means copying everything inside it.

Q14. When do cp and mv ask before overwriting?

Only when -i is given, or when the shell adds -i automatically through an alias (common for the root user on Red Hat family systems). Plain cp and mv overwrite silently; -n never overwrites. Run alias cp or alias mv to see what your own shell does.

Q15. How do you verify that a copy worked?

ls -l on the destination to see the new file with its size and time, and cat the copy to compare its contents with the original.

Q16. How do you rename a file in Linux?

With mv: mv old.txt new.txt. Renaming is just moving to a new name in the same place.

Q17. What is the difference between cp and mv?

cp makes a second copy and keeps the original. mv relocates or renames, so the original is no longer at the old path.

Q18. How do you delete a file and a directory?

rm file deletes a file. rmdir deletes an empty directory. rm -r deletes a directory with everything inside it. There is no recycle bin.

Q19. What is the difference between rm -r and rmdir?

rmdir only works on empty directories and fails otherwise. rm -r removes a directory and all its contents without that protection, so it is much riskier.

Q20. What does rm -f do and why is it dangerous?

-f forces deletion: no prompts and no errors for missing files. Combined with -r on a wrong path it can destroy a lot of data instantly.

Q21. Why does case matter in Linux file names?

Notes.txt, notes.txt and NOTES.txt are three different files. A wrong capital letter gives "No such file or directory".

Q22. Why can ls not show a file that exists?

It may be hidden. Names starting with a dot are skipped by plain ls; use ls -a to see them.

Sources

Man pages (man7.org/linux/man-pages/man1/): `ls, cp, mv, rm, rmdir, mkdir, touch, mktemp` - .1.html

https://www.nokia.com/bell-labs/about/history/innovation-stories/50-years-unix/ (Unix, 1969)

https://www.fsf.org/history/ (GNU announced 27 Sep 1983, started Jan 1984)

https://www.textbookoflinux.com/references/torvalds1991.html (Torvalds post, Aug 1991)

https://linuxreviews.org/Notes_for_linux_release_0.01 (0.01 released 17 Sep 1991)

https://www.debian.org/doc/manuals/project-history/detailed.en.html (Debian, Aug 1993)

http://www.slackware.com/announce/1.0.php (Slackware 1.00, 16 Jul 1993)

https://lists.ubuntu.com/archives/ubuntu-announce/2004-October/000003.html (Ubuntu 4.10, 20 Oct 2004)

https://lwn.net/Distributions/ (Ubuntu based on Debian)

https://docs.fedoraproject.org/en-US/quick-docs/fedora-and-red-hat-enterprise-linux/ (Fedora and RHEL)

https://developers.redhat.com/articles/2022/10/07/move-apt-dnf-package-management (apt/.deb, rpm/dnf)

https://wiki.archlinux.org/title/Arch_Linux (Arch: rolling, pacman, DIY)

https://unix.stackexchange.com/questions/146441/copy-over-existing-files-without-confirmation (RHEL/CentOS set cp -i, mv -i, rm -i aliases for root)

https://serverfault.com/questions/617206/where-is-this-bash-shell-alias-being-configured (those aliases shown on a CentOS root shell)